Lab 2 — Gaussian Elimination

Friday, October 23

Last updated: October 23, 2026.

Pick **Octave or Python**. In Python, `import numpy as np` once at the top. Matrix multiplication in Python is `@`. The symbol `*` multiplies entry by entry.

Learning objectives

1. Enter A and b and build the augmented matrix $[A \mid b]$.
2. Match a row operation with its elementary matrix and left-multiply.
3. Reduce a system with nonzero pivots to echelon form and back-substitute by hand.
4. Solve a unique system in one line and check $Ax = b$.
5. Adjust the steps when a pivot is zero, and tell no solution from infinitely many solutions.

Math connections

Elementary row operations and elimination (Lecture 21). Reading an echelon form for one solution, none, or infinitely many (Lecture 22).

Key syntax (this lab)

Rows are separated by `;` in Octave and by inner `[...]` lists in Python. A column vector in Octave uses `;` between entries. `b` must be a column before you attach it to A.

Task	Octave	Python (NumPy)
Matrix	`A = [1 2; 3 4]`	`A = np.array([[1., 2],[3, 4]])`
Column b	`b = [5; 6]`	`b = np.array([5., 6])`
Product Ax	`A * x`	`A @ x`
Augmented $[A \mid b]$	`M = [A b]`	`M = np.column_stack((A, b))`
Left-multiply by E	`M = E * M`	`M = E @ M`
Unique solve	`x = A \ b`	`x = np.linalg.solve(A, b)`
Check	`A * x - b`	`A @ x - b`

`size(M)` in Octave and `M.shape` in Python give the dimensions of the matrix M.

Additional Resources: Lachniet Ch. 2 · Syntax · Resources.

Row operations, briefly

From Lecture 21. These three operations do not change the solution set, because each one can be undone:

1. Swap two rows.
2. Multiply a row by a nonzero number.
3. Add a multiple of one row to another row, and replace that other row.

The matrix for an operation is what you get by doing the same operation to I. Left-multiplying $[A \mid b]$ by that matrix performs the operation. For two equations, the matrices are:

Swap the two rows: $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

Multiply row 1 by $c \neq 0$, or multiply row 2 by $c \neq 0$:

$$\begin{bmatrix} c & 0 \\ 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 \\ 0 & c \end{bmatrix}.$$

Add c times row 1 to row 2, or add c times row 2 to row 1:

$$\begin{bmatrix} 1 & 0 \\ c & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & c \\ 0 & 1 \end{bmatrix}.$$

The extra number sits in the row you are changing, in the column of the row you copy from. For three equations, start from I_3 and edit that same spot. Adding 2 times row 1 to row 3 uses

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}.$$

A 3×3 system

$$\begin{aligned} x_1 + x_2 + x_3 &= 6 \\ 2x_1 + 3x_2 + x_3 &= 11 \\ x_1 + 2x_2 + 3x_3 &= 14. \end{aligned}$$

So

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 6 \\ 11 \\ 14 \end{bmatrix}, \qquad [A \mid b] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 3 & 1 & 11 \\ 1 & 2 & 3 & 14 \end{array}\right].$$

Octave

```
A = [1 1 1; 2 3 1; 1 2 3]
b = [6; 11; 14]
M = [A b]
size(M)
```

Python

```
import numpy as np
A = np.array([[1., 1, 1],
              [2, 3, 1],
              [1, 2, 3]])
b = np.array([6., 11, 14])
M = np.column_stack((A, b))
M.shape
```

The algorithm, while every pivot is nonzero

Work on the augmented matrix.

1. Start at the upper-left corner. That entry is the first pivot.
2. Clear every entry below it in the first column. For each lower row, the multiplier is

$$c = \frac{\text{that entry}}{\text{the pivot}}.$$

   Replace the lower row by (that row) minus c times the pivot row.

3. The elementary matrix for that replacement is I with $-c$ in the row you are changing, in the pivot column. Store it and left-multiply: the new augmented matrix replaces the old one.
4. Move to the next pivot: one row down and one column to the right. Clear everything below that pivot the same way.
5. Stop at **echelon form**: zeros under each pivot, and the pivots stepping down to the right.
6. **Back-substitute** from the bottom equation upward.

One elimination step on the computer

The $(2,1)$ entry is 2 and the pivot is 1, so $c = 2$. Replace row 2 by row 2 minus 2 times row 1:

$$E_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Octave: `E21 = [1 0 0; -2 1 0; 0 0 1]` then `M = E21 * M`

Python:

```
E21 = np.array([[1., 0, 0],
                [-2, 1, 0],
                [0, 0, 1]])
M = E21 @ M
```

The new augmented matrix is

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 1 & 2 & 3 & 14 \end{array}\right].$$

Continue by hand to echelon form

Clear the $(3,1)$ entry. It equals the pivot, so $c = 1$. Replace row 3 by row 3 minus row 1:

$$E_{31} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}, \qquad E_{31}M = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 0 & 1 & 2 & 8 \end{array}\right].$$

In code this is the same pattern: build `E31`, then `M = E31 * M` in Octave or `M = E31 @ M` in Python.

The first column below the pivot is now clear. The next pivot is the $(2,2)$ entry, which is 1. The entry below it is also 1, so $c = 1$. Replace row 3 by row 3 minus row 2:

$$E_{32} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}, \qquad E_{32}E_{31}E_{21}[A \mid b] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & 3 & 9 \end{array}\right].$$

That is echelon form. The three pivots are 1, 1, and 3, and each sits to the right of the one above it.

Back substitution

Read the echelon matrix as equations, starting at the bottom.

$$\begin{aligned} 3x_3 = 9 \quad &\Rightarrow \quad x_3 = 3, \\ x_2 - x_3 = -1 \quad &\Rightarrow \quad x_2 - 3 = -1 \quad \Rightarrow \quad x_2 = 2, \\ x_1 + x_2 + x_3 = 6 \quad &\Rightarrow \quad x_1 + 2 + 3 = 6 \quad \Rightarrow \quad x_1 = 1. \end{aligned}$$

So $x = (1, 2, 3)$.

If the computer finishes the reduction

The same idea can keep going until every pivot is 1 and every other entry in a pivot column is 0. Then the right-hand column is the solution and you read it off. From the echelon matrix above, the remaining operations are: multiply row 3 by $\frac{1}{3}$; add the new row 3 to row 2; add -1 times the new row 3 to row 1; add -1 times the new row 2 to row 1. Each one is still "build the elementary matrix, left-multiply." The augmented matrix becomes

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array}\right],$$

which is the same $x = (1, 2, 3)$.

Octave: `x = A \ b`

Python: `x = np.linalg.solve(A, b)`

Check the way you would by hand

Compute Ax and compare it with b. The difference $Ax - b$ should be the zero vector.

Octave

```
A * x
A * x - b
```

Python

```
A @ x
A @ x - b
```

Both should return $(6, 11, 14)$ and then $(0, 0, 0)$.

Three systems to solve

Each system below has exactly one solution, and the algorithm above never meets a zero pivot. On the ticket: write the elementary matrices, reach echelon form, back-substitute by hand, then get the same x with `A \ b` or `np.linalg.solve`. Check with $Ax - b$.

C2.1. Two unknowns.

$$\begin{aligned} x_1 + 2x_2 &= 7 \\ 3x_1 + x_2 &= 11. \end{aligned}$$

C2.2. Two unknowns.

$$x_1 + 3x_2 = 10$$
$$2x_1 + x_2 = 5.$$

C2.3. Three unknowns.

$$x_1 + x_2 + x_3 = 6$$
$$x_1 + 2x_2 + x_3 = 7$$
$$2x_1 + x_2 + 3x_3 = 14.$$

When the pivot is zero

The first algorithm assumed nonzero pivots.

If the pivot position is 0, look **down that column** for a nonzero entry. Swap the pivot row with that row, then continue. The elementary matrix is I with those two rows exchanged.

Example:

$$x_2 + x_3 = 3$$
$$x_1 + x_2 + x_3 = 4$$
$$x_1 + 2x_2 + 3x_3 = 7.$$

The upper-left entry is 0. Row 2 has a 1 in column 1, so swap row 1 and row 2:

$$\left[\begin{array}{ccc} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{array}\right] \left[\begin{array}{ccc|c} 0 & 1 & 1 & 3 \\ 1 & 1 & 1 & 4 \\ 1 & 2 & 3 & 7 \end{array}\right] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 4 \\ 0 & 1 & 1 & 3 \\ 1 & 2 & 3 & 7 \end{array}\right].$$

The upper-left pivot is now 1. Clear below it as before (row 3 minus row 1, then row 3 minus row 2) to reach

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 4 \\ 0 & 1 & 1 & 3 \\ 0 & 0 & 1 & 0 \end{array}\right].$$

Back-substitution gives $x_3 = 0$, $x_2 = 3$, $x_1 = 1$.

No solution, or infinitely many

If the pivot position is 0 and **every entry below it is also** 0, no swap creates a pivot in that column. Read the row you are on. Two patterns matter.

No solution. A row whose left side is all zeros and whose right-hand side is not zero cannot happen. Stop. The system has no solution.

$$\begin{array}{r} x_1 + x_2 = 2 \\ 2x_1 + 2x_2 = 5 \end{array} \quad \longrightarrow \quad \left[\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 0 & 1 \end{array}\right].$$

The last row says $0 = 1$.

Infinitely many solutions. A row of all zeros, **including** the right-hand side, is a redundant equation. Any unknown that never receives a pivot is **free**. If you never meet a $0 = c$ row with $c \neq 0$, the system is consistent and each free unknown produces infinitely many solutions.

$$\begin{array}{r} x_1 + x_2 = 2 \\ 2x_1 + 2x_2 = 4 \end{array} \quad \longrightarrow \quad \left[\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 0 & 0 \end{array}\right].$$

Here x_2 is free. Set $x_2 = t$ and back-substitute: $x_1 = 2 - t$, with t any real number.

`A \ b` and `np.linalg.solve(A, b)` are the one-line tool for a **unique** solution, the case where the algorithm produced a pivot in every column. When the echelon form says there is no solution, or a free variable, report that conclusion from the echelon form.